ITA0610-Machine Learning

LAB EXPERIMENTS

Vasantha Nithi D Y

192424137

Lab 7: MLP Backpropagation for XOR Problem

Aim

To implement a **Multi-Layer Perceptron (MLP)** using **backpropagation** to solve the XOR problem.

Algorithm

1. Import the required libraries.
2. Create the XOR input dataset and target values.
3. Initialize the weights and biases randomly.
4. Perform forward propagation to calculate the output.
5. Calculate the output error.
6. Perform backpropagation to update the weights and biases.
7. Repeat the training process for a fixed number of epochs.
8. Display the final predictions.

Program

```
# Lab 7: MLP Backpropagation for XOR Problem


import numpy as np


# XOR input and output
X = np.array([
   [0, 0],
   [0, 1],
   [1, 0],
   [1, 1]
])


y = np.array([
```

```
    [0],
    [1],
    [1],
    [0]
])

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Set random seed
np.random.seed(42)

# Initialize weights and biases
input_hidden_weights = np.random.uniform(size=(2, 2))
hidden_output_weights = np.random.uniform(size=(2, 1))

hidden_bias = np.random.uniform(size=(1, 2))
output_bias = np.random.uniform(size=(1, 1))

learning_rate = 0.5

# Train the MLP
for epoch in range(10000):

    # Forward propagation
    hidden_input = np.dot(X, input_hidden_weights) + hidden_bias
```

```
hidden_output = sigmoid(hidden_input)

final_input = np.dot(hidden_output, hidden_output_weights) + output_bias
predicted_output = sigmoid(final_input)

# Calculate error
error = y - predicted_output

# Backpropagation
output_delta = error * sigmoid_derivative(predicted_output)

hidden_error = np.dot(
    output_delta,
    hidden_output_weights.T
)

hidden_delta = hidden_error * sigmoid_derivative(hidden_output)

# Update weights and biases
hidden_output_weights += learning_rate * np.dot(
    hidden_output.T,
    output_delta
)

input_hidden_weights += learning_rate * np.dot(
    X.T,
    hidden_delta
)

output_bias += learning_rate * np.sum(
    output_delta,
```

```
        axis=0,
        keepdims=True
    )

    hidden_bias += learning_rate * np.sum(
        hidden_delta,
        axis=0,
        keepdims=True
    )

# Display predictions
print("Predicted Output:")

for i in range(len(X)):
    print(
        X[i],
        "->",
        round(predicted_output[i][0], 3)
    )
```

Sample Output

Predicted Output:

[0 0] -> 0.016

[0 1] -> 0.983

[1 0] -> 0.983

[1 1] -> 0.021

Result

Thus, the **Multi-Layer Perceptron was successfully implemented using backpropagation**, and it successfully learned the XOR pattern.